

# PacMan shootout: Pixel reinforcement learning versus state machines in a multi-agent environment.

## GROUP 14

|                     |                         |
|---------------------|-------------------------|
| Marceau Messaussier | Fernando Cossio Ramirez |
| 2003-08-05          | 1995-11-27              |
| marceaum@kth.se     | fecr@kth.se             |



## Abstract

The problem setting is a multi-agent two-team Pac-Man capture-the-flag style match under partial observability. Scoring is done by bringing opponent's food home. Each agent receives full opponent state when in Line of Sight (LoS) and sparse cues when out of view. Each agent has health that decreases when in LoS of an opponent agent, therefore formation tactics become an important factor for winning. Coordinating autonomous agents under partial observability, adversarial opposition, and real-time constraints is a fundamental open problem whose solutions have broad real-world relevance, spanning autonomous robotics, multi-agent security, and cooperative navigation in contested environments. This project addresses it through two contrasting paradigms: a classical state-machine design and an end-to-end reinforcement learning algorithm.

For the state-machine-based approach, a high-level decision maker dynamically switches each agent between offensive and defensive state machines depending on the game state. Furthermore, in order to handle formation keeping we also have a leader-follower formation state. While this modular design enables interpretable and tunable agent behavior, the combinatorial explosion of edge cases that must be handled explicitly motivates exploring an end-to-end alternative.

The end-to-end approach takes as input a stacked tensor of feature maps encoding obstacle layout, agent positions, and opponent positions. The action space was simply the normalized acceleration of the agent. The policy was trained using Proximal Policy Optimization (PPO).

The end-to-end approach was not submitted for competition. The state-machine approach delivered stable results, finishing either 5th or 6th out of 20 teams across all maps and both competitions.

## 1 Introduction

We discuss multi-agent Pac-Man capture-the-flag problem, why it is hard and useful problem, and how we approached it with two different strategies.

Researchers and engineers often need systems that choose actions when information is incomplete and other actors work toward conflicting goals. Games supply a controlled arena: the rules are fixed, success is measurable, and experiments are repeatable. That combination makes games a standard way to stress-test ideas before anyone risks them in the messier real world.

The setting is a symmetric two-team Pac-Man-style maze. On its own half of the map, an agent moves as a ghost that can eat opponent Pac-Men; on the opponent half, it moves as a Pac-Man that can collect pellets and power capsules. Pellets count toward score, double after the carrier crosses back into home territory. Getting caught before that return drops the cargo and sends the agent back to spawn, so scoring is not just “collect everything” but “collect and deliver home while avoiding being caught.” Partial observability increases the game difficulty: when no teammate has line of sight to a moving enemy, the game still signals approximate enemy locations, but those cues are noisy. Agents must therefore integrate planning, evasion, and teammate coordination under beliefs that aren’t always verifiable. To make it more interesting, the “Shootout” version of the game increases the complexity by adding a health layer and guns; every agent inflicts health damage on the closest opponent within LoS. The damage is symmetric and proportional to the closeness of the two opponents. To break the symmetry and stay alive, formations of more than one agent can be used to superpose the damage inflicted towards one opponent.

Solving such problems well would make it easier to field teams of co-operating agents — warehouse robots, search-and-rescue units, or any fleet that must plan under partial sensing with adversarial pressure, especially in environments where tight inter-agent coordination is critical to survival.

We approach the problem with two completely different strategies that make opposite bets on engineering risk. One approach uses grid A\* pathfinding as a basis, the other relies entirely on deep reinforcement learning methods. The first approach uses a high level decision maker that switches among hand-designed behaviors (attackers and defenders); it aims for a decent performance in competition that mitigates the risk of not having a submittable learned policy. The second approach follows an Atari-style Reinforcement Learning setting: we encode the game state in a multi-channel tensor and train a deep reinforcement learner to predict the next action based on the pixels. We finished the training and integration pipeline, but the time budget did not allow for enough iterations to reach competitive results on the learned policy. The same rules therefore expose what more traditional planning and heuristics can deliver in a short time horizon, versus how much time it takes to first set up the training infrastructure and to do the learn-

ing iterations.

### 1.1 Contribution

Our contribution is twofold. For the state machines’ part, we develop interesting tools such as a precomputed LoS map that allows the behaviors to predict more vulnerable locations in the map. For the end-to-end approach, we build a training infrastructure that makes it possible to train CNN-based policies using widespread tools for multi-agent reinforcement learning such as PettingZoo and AgileRL.

On the state machine side, we develop a set of tools enabling accurate world-state estimation under partial observability. A precomputed Line-of-Sight (LoS) grid allows behaviors to reason about map vulnerabilities and anticipate exposure risk. Pathfinding relies on a precomputed Dijkstra grid, while local motion is handled by a Velocity Obstacle (VO) controller. Together, these components underpin a library of modular behaviors that compose into the offensive and defensive state machines driving each agent.

On the end-to-end reinforcement learning approach we show that it is possible to learn policies based on pixels through a setup that subclasses the Game Manager to send protobuf messages over http to a PettingZoo python environment which is further used to train PPO using Torch.

### 1.2 Outline

The report is organized as follows. Section 2 reviews the algorithmic building blocks underlying both approaches, from navigation and collision avoidance to formation control, Voronoi decomposition, and reinforcement learning. Section 3 details the three-layer state-machine architecture and the end-to-end PPO training pipeline. Section 4 presents the competition results, compares our approach against the top-performing groups, and analyzes our limitations. Section 5 summarizes our findings and outlines directions for future work.

## 2 Related work

This section reviews the key techniques underlying our two approaches. We first cover the building blocks of the state-machine approach: shortest-path computation for navigation, velocity obstacles and artificial potential fields for collision avoidance and threat response, a pursuit model for enemy interception, and crowd-inspired formation forces for coordinated group behavior. We then introduce the reinforcement learning background that underpins the end-to-end approach.

To enable efficient navigation, we equip our agents with a notion of distance and direction by pre-computing pairwise shortest-path distances

on the grid using Dijkstra’s algorithm [2]. Running Dijkstra from every cell yields complete distance grids, enabling constant-time distance queries during gameplay.

At the control level, agents must avoid collisions with teammates while remaining reactive to enemies. We use the Velocity Obstacle (VO) framework [4], which identifies the set of velocities leading to future collisions and steers agents outside it. While well-suited for real-time multi-agent navigation, VO alone proves insufficient in adversarial settings where opponents behave unpredictably.

Since VO relies on predicted trajectories, it can produce unsafe decisions when opponents deviate unexpectedly or when we are being chased. We therefore complement it with Artificial Potential Fields (APF) [5], which repel agents from nearby enemies and attract them toward safe directions. This component is not always active: depending on the game state, agents may instead pursue enemies to drive them out of home territory. In that case, pursuit is achieved by predicting the future position of the target under a constant-velocity assumption, following Reynolds’ seek/pursuit steering model [7], where a steering force is generated toward the estimated position  $d = v_{max} \cdot \Delta t$  ahead.

The behaviors described so far operate at the individual agent level. To enable coordinated group behavior, we introduce two complementary mechanisms operating at different stages of the game.

On the defensive side, we introduce a *Follower* state within the defense state machine, where designated agents act as bodyguards to a self-assigned master. Followers maintain their formation through a combination of attraction and repulsion forces based on distance, and align their speed to that of the master to ensure reactive, cohesive movement. This approach is inspired by the crowd locomotion model of [1], which combines distance-based attraction, repulsion, and velocity alignment. Coupled with the VO controller, this formation remains stable even in narrow corridors.

On the offensive side, coordination is achieved through a global map decomposition rather than explicit agent-to-agent forces. Once enemy positions are estimated, we compute a Voronoi decomposition of the map to assess territorial control. Each cell is assigned to whichever agent (ally or opponent) can reach it via the shortest path, yielding a partition of the environment into regions of influence. We build on the graph Voronoi construction of [3], but rather than running a dedicated multi-source Dijkstra, we reuse our precomputed distance grids, reducing each Voronoi update to a simple argmin lookup over all agents.

While this state-machine-based approach offers interpretable and modular agent behavior, the considerable number of parameters and strategies to tune motivated us to explore a complementary end-to-end reinforcement learning approach.

Reinforcement learning provides another way to build game-playing agents

by learning behavior from interaction with the environment rather than explicitly coding every decision rule. Sutton and Barto [9] describe the standard reinforcement learning framework, where an agent learns a policy by taking actions based on a state and obtaining rewards over time. This motivated our separate reinforcement learning attempt, which was explored as an alternative to the hand-designed planning system where the state was encoded as a 2D 11-channel pixel array.

Proximal Policy Optimization (PPO) [8] is a reinforcement learning policy optimization algorithm that trains a policy by alternating between interacting with the environment and improving a “surrogate” objective function using stochastic gradient ascent. It further clips the new policy to be similar to the previous step to avoid auto-destructive policies. We use it in our second approach as the optimization objective.

Multi-agent deep deterministic policy gradient (MADDPG) [6] addresses the increased variance that is created by the increased number of agents. It uses centralized training with decentralized execution: each agent’s critic conditions on the joint actions (and optionally policies) of all agents during training, but each actor uses only local observations at execution time, so the method applies to cooperative, competitive, and mixed environments.

### 3 Proposed Method

In this section we describe our two parallel approaches: the state-machine-based approach and the end-to-end reinforcement learning approach.

**State machine approach.** Our solution is organized around three layers: a central command unit that estimates the world state and assigns roles, a set of per-agent state machines that govern high-level behavior, and a low-level motion controller that executes the resulting movement commands. The state machines cover offensive actions such as food collection and evasion, as well as defensive behaviors including a formation-keeping mechanism where agents coordinate around a designated leader.

At the top layer, the central command unit is responsible for two tasks: estimating a world state from partial observations, and issuing role assignments (attacker or defender) to all agents accordingly. To support world-state estimation, the command unit precomputes a Line-of-Sight (LoS) grid at match startup by performing four raycasts per cell pair, one from each corner, ensuring that agents do not incorrectly consider themselves concealed when partially exposed. During the match, the command unit aggregates the observations of all allies to detect enemies and maintains their last known positions. This allows the correct LoS grid to be retrieved for any estimated enemy location. Role assignments are then recomputed whenever a significant change in game state is detected.

At the behavior layer, each agent runs an independent state machine conditioned on its assigned role. The attacker state machine governs how agents engage enemy territory, as depicted in Figure 1 on the left side. The defender state machine governs how agents protect their own side, as shown in Figure 1 on the right side.

The attack state machine computes paths to avoid enemies both on state entry and periodically during execution, typically every 0.5 seconds, ensuring reactivity to changes in enemy positions. Pill selection is guided by the Voronoi decomposition as described in [3]: each attacker is assigned to the region of the map it controls, directing it toward the food clusters most accessible from its current position and reducing overlap between allied attackers.

The defense state machine includes several notable behaviors. The *Go to Cluster* behavior identifies high-density food regions by computing, for each pill, the number of neighboring pills within a radius of 1.5 cells, and prioritizes clusters that are close to the border. The *Go to Border* behavior selects an approach position that is occluded by obstacles, so that the agent is not exposed upon crossing into enemy territory. The *Hide from Enemies* behavior queries the LoS grid maintained by the central command unit to identify the least exposed cell currently available.

The defense state machine also includes a *Follower* state, in which designated agents act as bodyguards to a self-assigned master. Followers apply attraction, repulsion, and speed alignment forces following the flocking model of [1], maintaining a coherent formation without explicit coordination. The VO controller ensures smooth collective motion even in narrow corridors.

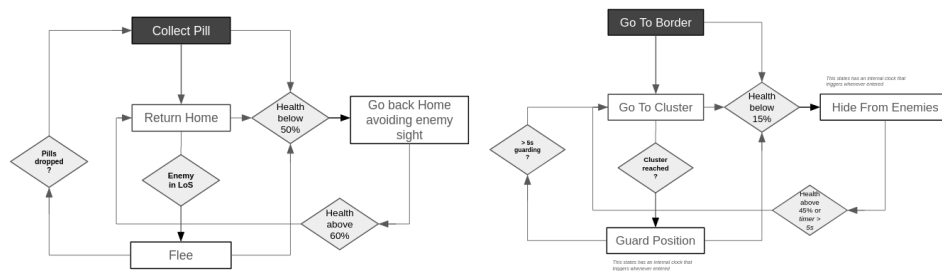


Figure 1: State machines: on the left the attack state machine and on the right the defense state machine. The black state is the initial state, all transitions are represented with arrows and conditions with diamonds.

At the motion layer, agents navigate by following paths computed from Dijkstra distance grids precomputed by the central command unit, advancing step by step toward the next node. Local motion is then handled by a Velocity Obstacle (VO) controller [4] augmented with Artificial Potential

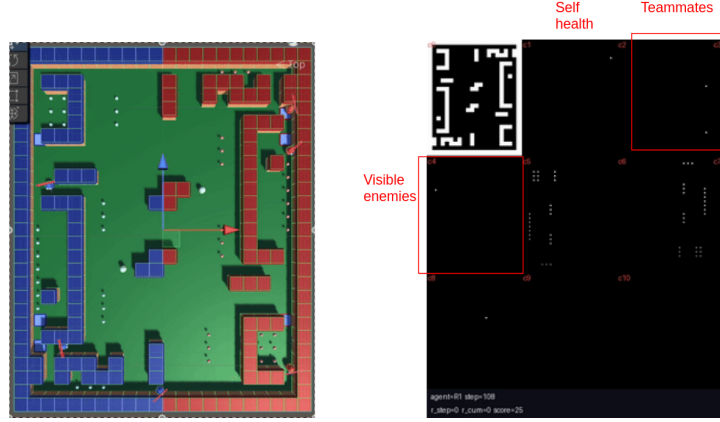


Figure 2: Flattened channels of the observation space (right) with the correspondence in the game environment (left).

Fields (APF) [5]. The APF component is adaptive: it applies a repulsive force when an enemy should be avoided, and switches to a pursuit behavior following Reynolds’ steering model [7] when an enemy comes within capture range.

**End-to-end reinforcement learning approach.** Our end-to-end approach is organized around three components: a training infrastructure that connects the game engine and standard RL tooling, an observation and action space that encodes the environment as a pixel tensor, and a PPO training loop that optimizes a CNN-based policy.

At the infrastructure layer, the game engine is extended through a subclass of the Game Manager, `RLTrainGameManager`, which exposes the simulation as an HTTP server communicating via Protocol Buffer messages over three endpoints: `/initialize`, `/reset`, and `/step`. Physics advancement is driven entirely by the Python client through explicit `/step` calls, giving full control over simulation speed during training. On the Python side, each game instance is wrapped into a PettingZoo parallel environment, augmented with frame stacking and frame skipping via SuperSuit, and several server instances are run in parallel using an `AsyncPettingZooVecEnv`.

At the observation layer, the state is represented as a multi-channel tensor of shape  $C \times H \times W$ , where each channel encodes a distinct spatial feature of the map: obstacle occupancy, ally positions, opponent positions, and food layout. See Figure 2. The action space is a continuous 2D acceleration vector, normalized to  $[-1, 1]$  on each axis, applied directly to the agent’s physics body.

At the training layer, two independent shared-policy PPO agents are maintained, one per team, each assigned to a dedicated GPU.

## 4 Experimental Results

We evaluate our system across two competitions, each played on two maps, against 19 other teams. Our solution ranked 6th on Map F3 and 5th on Map F12 in the first competition, and 6th on both Map F3 and Map F12 in the second competition, reflecting stable performance across maps and agent counts. In the following, we describe the experimental setup, present the full competition results, and analyze the causes of our performance pattern and the strategies that distinguished the top-performing teams. Eventually, we also dive into our RL approach and explore why we couldn't send it.

### 4.1 Experimental Setup

Both competitions were held one week apart and evaluated all 20 teams on the same two maps: Map F3 and Map F12. Both maps share a symmetric maze layout designed to be deep enough to make death costly. The main difference between the two maps lies in the number of agents: Map F3 involved 6 agents per team, while Map F12 scaled up to 12, which naturally opens the door to different team strategies. Each group ran their solution independently on their own hardware and submitted a single run per competition. Each team played 10 matches on the computer of the home team, and the 9 remaining matches on the opponents' computer. Victories are counted as 3 Victory Points (VP), draws are counted as 1 VP and defeats are counted as 0 VP.

### 4.2 Analysis of Outcome

This section analyzes our results across both approaches. We first report the competition outcomes of the state-machine solution, then present the training results of the reinforcement learning policy. We then compare our design against the top-performing groups and identify the structural weaknesses that limited our ceiling.

**State Machine.** Our group (G14) ranked 4th out of 20 groups on the combined VP across both maps and both competitions, with consistent improvement from the first to the second competition across all maps, as shown in Table 1. In this section we analyze the strategies of the top-performing groups, draw comparisons with our own approach, and identify the limitations that prevented us from reaching the podium.

**End-to-end pixel based reinforcement learning** . Even if very interesting, our most successful training with PPO still collapsed to only drive the agents to the food on the opposite corner and never really learned to bring the food back. An interesting emergence of adversarial strategies is



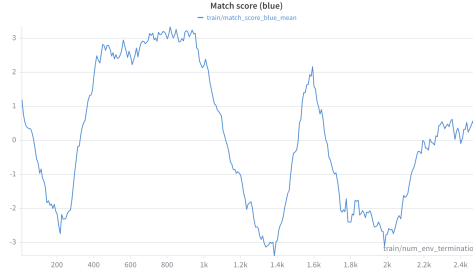


Figure 3: Score delta over training matches as seen from the blue policy.

shown on 3 where the winner oscillates between the two teams. A live GIF sampled every two full training matches can be viewed in the [weights and biases run](#).

#### 4.2.1 Comparison with top-performing groups.

Two of the groups that ranked above us G19 and G11 in the ranking presented in Figure 4 and the winner of the final competition G08 each adopted solutions that share several design principles with ours, but differed in key tactical decisions.

Group G08 is the closest to our architecture: they combine Velocity Obstacles for traffic management, shared enemy tracking across agents, dynamic attacker/defender role switching, and cooperative coalition formation. Their most distinctive feature is their use of danger-weighted A\* for pathfinding, which bakes enemy exposure directly into the path cost. Our approach differs in that we precompute exhaustive pairwise Dijkstra grids at match startup, which enables constant-time distance queries but does not account for enemy positions by default. We partially compensate for this by computing dynamic Dijkstra grids that penalize enemy LoS on demand, for instance during retreat, but this remains a sparse fallback rather than a systematic policy as it seems to be in G08.

Group G19 shares most of the design principles described above, but differentiates itself through a stronger focus on coordinated formations and carefully chosen defensive positions along the border.

Group G11 differs slightly from us because of their controller, they do not use (VO). Their LoS handling is also more complete than ours: they precompute a combined penalty heatmap that fuses a static 'killzone' (covered position near the border) with a dynamic LoS layer (how many enemies currently see it). Our approach handles these two mechanisms separately, which is interpretable but more complex to deal with. Their defender and attacker logic is otherwise very similar to ours.

#### 4.2.2 Limitations of our approach.

Despite our stable ranking, several structural weaknesses limited our ceiling. As visible on the right side of Figure 4, our score delta is surprisingly low compared to the top groups, falling short by a factor of 3 to 4. This suggests that our defense was consistent enough to prevent losses, but our offensive output was severely lacking. Our attackers typically operate independently, making them individually vulnerable to interception, whereas the top groups systematically use strategic defensive positions behind walls that could handle our lonely attackers. The occasional inconsistency in cluster assignment further weakened our defensive coordination, leaving the defenders protecting suboptimal food regions.

Our formation-keeping mechanism, while theoretically motivated, underperformed in practice. Followers sometimes cannot join the master as he is too fast, resulting in formations that visually disperse and fail to confer the intended group advantage. Furthermore, followers sometimes prefer to remain exposed rather than push the master toward cover. Addressing these issues would require tighter coupling between the follower and the master dynamics.

| Group        | Comp1 |       |     |       | Comp2 |       |     |       | Total |       |
|--------------|-------|-------|-----|-------|-------|-------|-----|-------|-------|-------|
|              | F3    |       | F12 |       | F3    |       | F12 |       | VP↑   | Score |
|              | VP    | Score | VP  | Score | VP    | Score | VP  | Score |       |       |
| A4G19        | 57    | 764   | 54  | 805   | 48    | 649   | 54  | 817   | 213   | 3035  |
| A4G11        | 57    | 673   | 39  | 515   | 48    | 714   | 45  | 648   | 189   | 2550  |
| A4G02        | 46    | 522   | 43  | 343   | 42    | 497   | 36  | 418   | 167   | 1780  |
| <b>A4G14</b> | 31    | 130   | 40  | 216   | 42    | 304   | 42  | 214   | 155   | 864   |
| A4G05        | 31    | 335   | 41  | 466   | 24    | 27    | 45  | 378   | 141   | 1206  |
| A4G10        | 40    | 339   | 28  | 19    | 39    | 243   | 33  | 44    | 140   | 645   |
| A4G06        | 39    | 350   | 40  | 297   | 27    | -63   | 21  | -260  | 127   | 324   |
| A4G03        | 15    | -343  | 14  | -281  | 45    | 421   | 51  | 416   | 125   | 213   |
| A4G08        | 15    | -40   | 11  | -233  | 51    | 684   | 48  | 665   | 125   | 1076  |
| A4G04        | 46    | 626   | 42  | 598   | 15    | -520  | 18  | -399  | 121   | 305   |
| A4G16        | 29    | 48    | 18  | -84   | 33    | 346   | 36  | 517   | 116   | 827   |
| A4G07        | 27    | -82   | 15  | -380  | 36    | 288   | 27  | 110   | 105   | -64   |
| A4G20        | 31    | 289   | 27  | 112   | 15    | -445  | 24  | -182  | 97    | -226  |
| A4G17        | 26    | -31   | 28  | -83   | 18    | -70   | 24  | -45   | 96    | -229  |
| A4G09        | 30    | -29   | 28  | 145   | 18    | -232  | 6   | -268  | 82    | -384  |
| A4G13        | 33    | 418   | 39  | 408   | 6     | -1099 | 3   | -1133 | 81    | -1406 |
| A4G12        | 14    | -5    | 7   | -93   | 27    | 120   | 16  | -39   | 64    | -17   |
| A4G01        | 6     | -948  | 2   | -1097 | 21    | -200  | 21  | -80   | 50    | -2325 |
| A4G18        | 22    | -417  | 13  | -876  | 3     | -1129 | 3   | -1355 | 41    | -3777 |
| A4G15        | 8     | -794  | 8   | -797  | 9     | -535  | 13  | -466  | 38    | -2592 |

Table 1: A4 competition results.

Table 1 shows the per-map VP and sum of score deltas for all groups on both maps. Our group (G14) ranked 4th out of 20 groups on the sum of VP of both maps and both competitions.

Figure 4 shows that group A4G14 improved the score across all maps from the first competition to the second competition

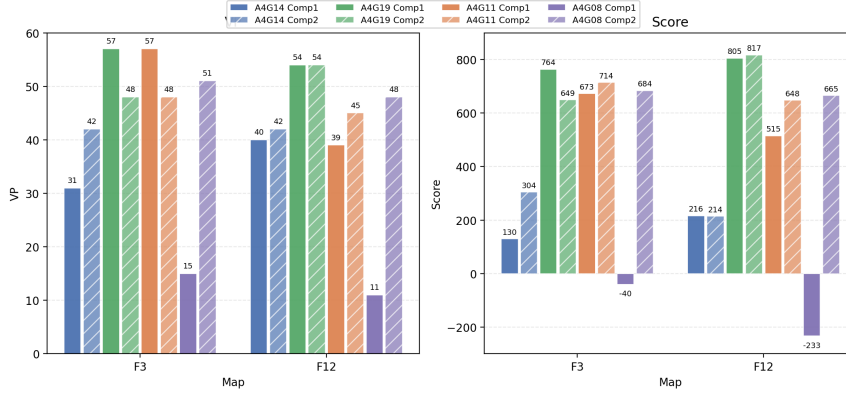


Figure 4: A4G14 comparison across competitions versus top scoring teams and underdog A4G08 that won the live final tournament for map F3.

## 5 Summary and Conclusions

This project investigated two contrasting approaches to multi-agent coordination under partial observability and adversarial pressure: a modular state-machine system and an end-to-end reinforcement learning pipeline.

The state-machine approach delivered consistent and competitive results, ranking 4th in total Victory Points across all maps and both competitions. Our key contributions, the precomputed LoS grid, the Dijkstra-based navigation, and the flocking-inspired formation mechanism, enabled interpretable and tunable agent behavior. However, the approach was limited by fragile offensive coordination, with attackers operating independently and remaining vulnerable to interception, and by an underperforming formation mechanism where followers failed to reliably maintain proximity to the master.

The end-to-end approach demonstrated that training CNN-based policies directly within the game engine is feasible through our PettingZoo/PPO infrastructure, but the engineering overhead consumed most of the available time budget, leaving too few training iterations for the policy to reach a competitive level.

Looking forward, hierarchical reinforcement learning appears as a natural next step: high-level strategic decisions could be handled by a learned policy, while low-level execution remains delegated to the hand-designed behaviors. This would bridge the gap between our two approaches and maybe accelerate drastically the learning speed of the RL algorithm.

## 6 Usage of LLMs

LLMs (Opus 4.7, GPT 5.5, Composer 2.5) were used by the authors to edit code through the IDE Cursor (meaning they edited code sections independently based on prompts). Opus and Composer were used during the creation of the report within the course guidelines.

## References

- [1] Emiliano Cristiani, Marta Menci, Marco Papi, and Leonard Brafman. An all-leader agent-based model for turning and flocking birds. *arXiv preprint arXiv:2010.01990*, 2021.
- [2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [3] Martin Erwig. The graph voronoi diagram with applications. *Networks*, 36(3):156–163, 2000.
- [4] Paolo Fiorini and Zvi Shiller. Motion planning in dynamic environments using velocity obstacles. *The International Journal of Robotics Research*, 17(7):760–772, 1998.
- [5] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986.
- [6] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments, 2020.
- [7] Craig W. Reynolds. Steering behaviors for autonomous characters. *Game Developers Conference Proceedings*, 1999.
- [8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.
- [9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2 edition, 2018.